

A decorative graphic on the left side of the slide. It consists of a blue parallelogram and a light green parallelogram, both tilted at an angle. The blue shape is in the foreground, and the green shape is partially behind it. They are set against a dark blue background with faint, lighter blue diagonal stripes.

RUSTikales Rust for beginners



Plan for today



Plan for today

1. Recap



Plan for today

1. Recap
2. Functions



Plan for today

1. Recap
2. Functions
3. Floating Point Numbers



1. Recap



1. Recap

- `loop {}` to create an infinite loop
- `while condition {}` to create a conditional loop
- `for elem in collection {}` to create an iterator over a collection



1. Recap

- `loop {}` to create an infinite loop
- `while condition {}` to create a conditional loop
- `for elem in collection {}` to create an iterator over a collection
- **Ownership-Model**
 - Every value in Rust has **exactly one owner**
 - **Values are dropped** (memory is freed) **when the owner is dropped**
 - **Ownership-Conflicts** are resolved by **moving ownership**, if we can't copy the value



1. Recap

- `loop {}` to create an infinite loop
- `while condition {}` to create a conditional loop
- `for elem in collection {}` to create an iterator over a collection
- Ownership-Model
- References
 - Way to `borrow data without moving or copying it`
 - Can be immutable or mutable



1. Recap

- `loop {}` to create an infinite loop
- `while condition {}` to create a conditional loop
- `for elem in collection {}` to create an iterator over a collection
- Ownership-Model
- References
- Borrow Checker
 - zero mutable references to a value → infinite immutable references allowed
 - one mutable reference to a value → zero immutable references allowed
 - two+ mutable references to a value → forbidden, compiler error



1. Recap

- `loop {}` to create an infinite loop
- `while condition {}` to create a conditional loop
- `for elem in collection {}` to create an iterator over a collection
- Ownership-Model
- References
- Borrow Checker
- Lifetimes
 - Every reference has a lifetime
 - Reference must be **valid between creation and usage**
 - Value it points to must be alive
 - Used by the Borrow Checker

1. Recap

```
fn main() {  
    let a: i32 = 10;  
    let b: &i32 = &a;  
    println!("{}", *b);  
}
```

1. Recap

```
fn main() {  
    let a: i32 = 10;  
    let b: &i32 = &a; Immutable borrow  
    println!("{}", *b);  
}
```

1. Recap

```
fn main() {  
    let a: i32 = 10;  
    let b: &i32 = &a;  
    println!("{}", *b);  
}
```

Type: Reference to i32

1. Recap

```
fn main() {  
    let a: i32 = 10;  
    let b: &i32 = &a;  
    println!("{}", *b);  
}
```

Dereference `b` to get the original value



1. Recap

Addr	Stack
------	-------

```
fn main() {  
    let a: i32 = 10;  
    let b: &i32 = &a;  
    println!("{}", *b);  
}
```

1. Recap

Addr	Stack	
0x4000	a	10

```
fn main() {  
    let a: i32 = 10;  
    let b: &i32 = &a;  
    println!("{}", *b);  
}
```

1. Recap



Addr	Stack	
0x4000	a	10
0x4004	b	0x4000

```
fn main() {  
    let a: i32 = 10;  
    let b: &i32 = &a;  
    println!("{}", *b);  
}
```

1. Recap

Addr	Stack	
0x4000	a	10
0x4004	b	0x4000

Read the value at that memory location

```
fn main() {  
    let a: i32 = 10;  
    let b: &i32 = &a;  
    println!("{}", *b);  
}
```

1. Recap

Addr	Stack	
0x4000	a	10
0x4004	b	0x4000

```
fn main() {  
    let a: i32 = 10;  
    let b: &i32 = &a;  
    println!("{}", *b);  
}
```

10

1. Recap

```
fn main() {  
    let a: u8 = 12;  
    let r1: &u8 = &a;  
    let r2: &u8 = &a;  
    println!("r1 = {}", r1);  
    println!("r2 = {}", r2);  
}
```

1. Recap

```
fn main() {  
    let a: u8 = 12;  
    let r1: &u8 = &a;  
    let r2: &u8 = &a;  
    println!("r1 = {}", r1);  
    println!("r2 = {}", r2);  
}
```

Immutable borrows

1. Recap

```
fn main() {  
    let a: u8 = 12;  
    let r1: &u8 = &a;  
    let r2: &u8 = &a;  
    println!("r1 = {}", r1);  
    println!("r2 = {}", r2);  
}
```

Implicit dereference



1. Recap

Addr	Stack
------	-------

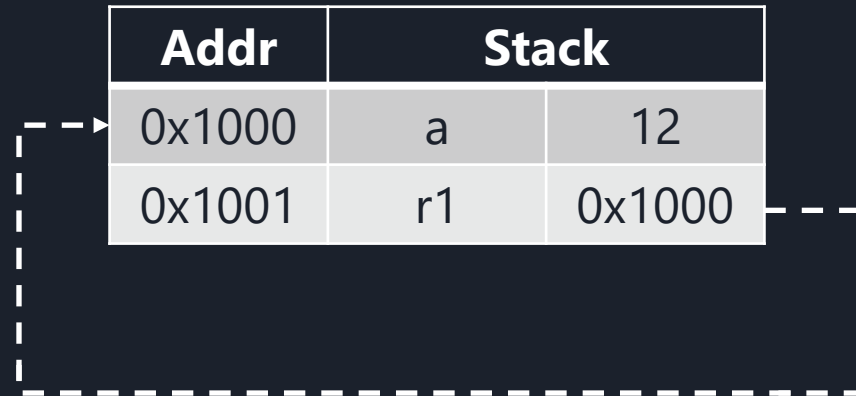
```
fn main() {  
    let a: u8 = 12;  
    let r1: &u8 = &a;  
    let r2: &u8 = &a;  
    println!("r1 = {}", r1);  
    println!("r2 = {}", r2);  
}
```

1. Recap

Addr	Stack	
0x1000	a	12

```
fn main() {  
    let a: u8 = 12;  
    let r1: &u8 = &a;  
    let r2: &u8 = &a;  
    println!("r1 = {}", r1);  
    println!("r2 = {}", r2);  
}
```

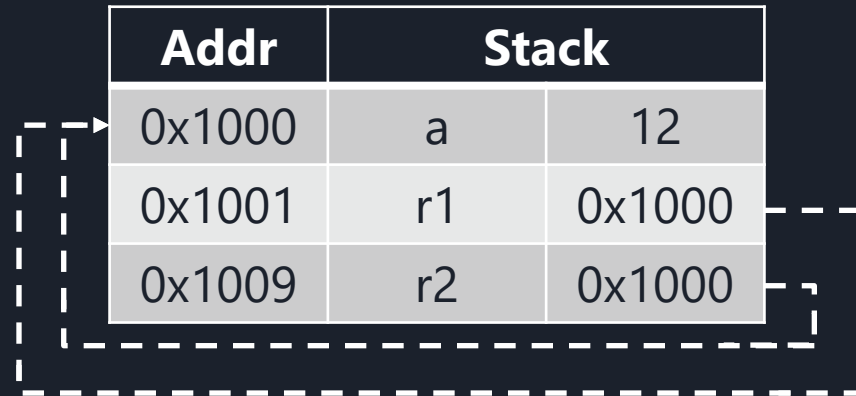
1. Recap



Addr	Stack	
0x1000	a	12
0x1001	r1	0x1000

```
fn main() {  
    let a: u8 = 12;  
    let r1: &u8 = &a;  
    let r2: &u8 = &a;  
    println!("r1 = {}", r1);  
    println!("r2 = {}", r2);  
}
```

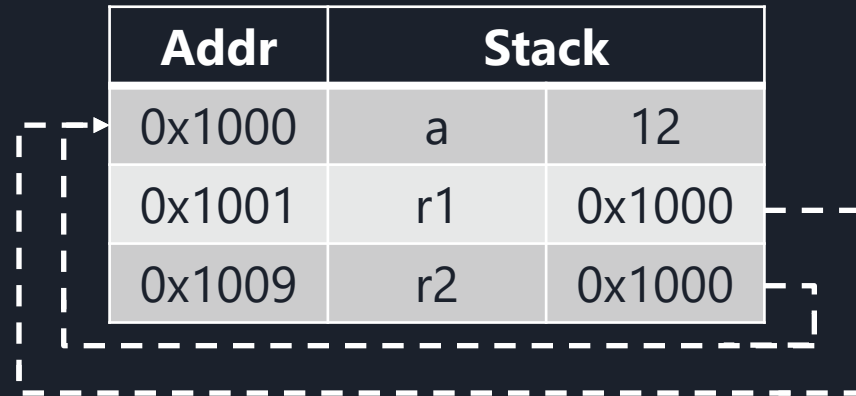
1. Recap



Addr	Stack	
0x1000	a	12
0x1001	r1	0x1000
0x1009	r2	0x1000

```
fn main() {  
    let a: u8 = 12;  
    let r1: &u8 = &a;  
    let r2: &u8 = &a;  
    println!("r1 = {}", r1);  
    println!("r2 = {}", r2);  
}
```

1. Recap



A diagram illustrating a stack frame. It consists of a table with three rows and two columns: 'Addr' and 'Stack'. The first row shows '0x1000' pointing to 'a' with value '12'. The second row shows '0x1001' pointing to 'r1' with value '0x1000'. The third row shows '0x1009' pointing to 'r2' with value '0x1000'. A dashed box encloses the entire table, and an arrow points from the left to the first row.

Addr	Stack
0x1000	a 12
0x1001	r1 0x1000
0x1009	r2 0x1000

```
fn main() {  
    let a: u8 = 12;  
    let r1: &u8 = &a;  
    let r2: &u8 = &a;  
    println!("r1 = {}", r1);  
    println!("r2 = {}", r2);  
}
```

`println!` automatically dereferences for us

1. Recap



Addr	Stack	
0x1000	a	12
0x1001	r1	0x1000
0x1009	r2	0x1000

```
fn main() {  
    let a: u8 = 12;  
    let r1: &u8 = &a;  
    let r2: &u8 = &a;  
    println!("r1 = {}", r1);  
    println!("r2 = {}", r2);  
}
```

`println!` automatically dereferences for us

r1 = 12
r2 = 12

1. Recap

```
fn main() {  
    let mut a: i16 = 420;  
    let b: &mut i16 = &mut a;  
    *b = 1337;  
    println!("a = {}", a);  
}
```

1. Recap

```
fn main() {  
    let mut a: i16 = 420;  
    let b: &mut i16 = &mut a; Mutable borrow  
    *b = 1337;  
    println!("a = {}", a);  
}
```

1. Recap

```
fn main() {  
    let mut a: i16 = 420;  
    let b: &mut i16 = &mut a;  
    *b = 1337;  
    println!("a = {}", a);  
}
```

Mutable borrow only possible when value itself is mutable

Mutable borrow



1. Recap

Addr	Stack
------	-------

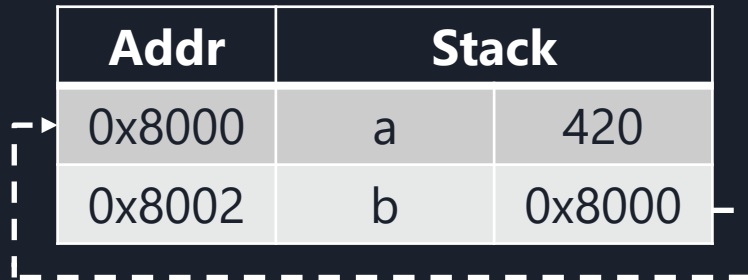
```
fn main() {  
    let mut a: i16 = 420;  
    let b: &mut i16 = &mut a;  
    *b = 1337;  
    println!("a = {}", a);  
}
```

1. Recap

Addr	Stack	
0x8000	a	420

```
fn main() {  
    let mut a: i16 = 420;  
    let b: &mut i16 = &mut a;  
    *b = 1337;  
    println!("a = {}", a);  
}
```

1. Recap



Addr	Stack	
0x8000	a	420
0x8002	b	0x8000

```
fn main() {  
    let mut a: i16 = 420;  
    let b: &mut i16 = &mut a;  
    *b = 1337;  
    println!("a = {}", a);  
}
```

1. Recap



Addr	Stack	
0x8000	a	420
0x8002	b	0x8000

A dashed white box encloses the two rows of the table. An arrow points from the left to the first row.

Write 1337 into location
pointed to by b

```
fn main() {  
    let mut a: i16 = 420;  
    let b: &mut i16 = &mut a;  
    *b = 1337;  
    println!("a = {}", a);  
}
```

1. Recap

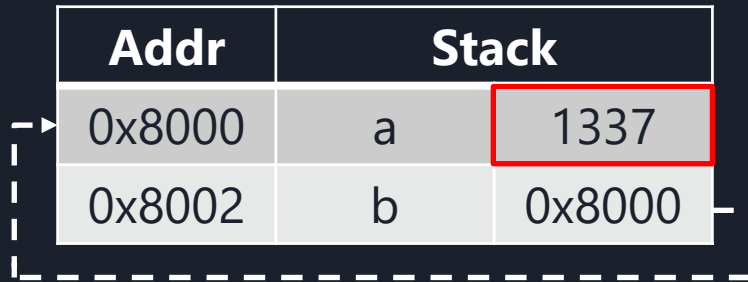


Addr	Stack	
0x8000	a	420
0x8002	b	0x8000

Write 1337 into location
pointed to by b

```
fn main() {  
    let mut a: i16 = 420;  
    let b: &mut i16 = &mut a;  
    *b = 1337;  
    println!("a = {}", a);  
}
```

1. Recap

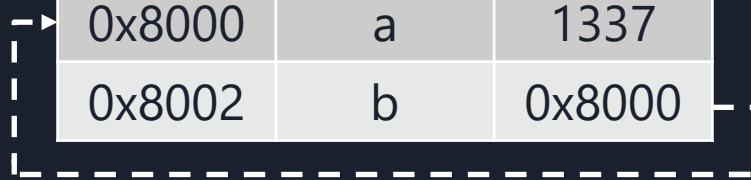


Addr	Stack	
0x8000	a	1337
0x8002	b	0x8000

Write 1337 into location
pointed to by b

```
fn main() {  
    let mut a: i16 = 420;  
    let b: &mut i16 = &mut a;  
    *b = 1337;  
    println!("a = {}", a);  
}
```

1. Recap



Addr	Stack	
0x8000	a	1337
0x8002	b	0x8000

A dashed white box encloses the two rows of the table. A dashed white arrow points from the left edge of the box to the first row.

a = 1337

```
fn main() {  
    let mut a: i16 = 420;  
    let b: &mut i16 = &mut a;  
    *b = 1337;  
    println!("a = {}", a);  
}
```



2. Functions



2. Functions

- Programming languages come with **many control flow abstractions**



2. Functions

- Programming languages come with many control flow abstractions
 - if
 - loops, break, continue



2. Functions

- Programming languages come with **many control flow abstractions**
- Functions are another way of controlling your code



2. Functions

- Programming languages come with **many control flow abstractions**
- Functions are another way of controlling your code
- Functions allow us to **isolate pieces of logic**, instead of writing one big blob of code



2. Functions

- Programming languages come with **many control flow abstractions**
- Functions are another way of controlling your code
- Functions allow us to **isolate pieces of logic**, instead of writing one big blob of code
 - We can write a **function that calculates prime numbers**
 - We can write a **function that checks if a vector contains a given number**
 - We can write a **function that checks if a vector only contains prime numbers**



2. Functions

- Programming languages come with **many control flow abstractions**
- Functions are another way of controlling your code
- Functions allow us to **isolate pieces of logic**, instead of writing one big blob of code
- Functions in programming are almost* identical to mathematical functions
 - They **take in some inputs**
 - They **do something** with those inputs
 - They **return some output**
 - In programming, functions can have **side effects**
 - Writing to a file, printing to the console, ...



2. Functions

- Programming languages come with **many control flow abstractions**
- Functions are another way of controlling your code
- Functions allow us to **isolate pieces of logic**, instead of writing one big blob of code
- Functions in programming are almost* identical to mathematical functions
- Functions are declared using the keyword **fn**
 - The most important one – **fn main()** – always needs to be declared



2. Functions

```
fn square(x: i16) -> i16 {  
    return x * x;  
}
```



2. Functions

Parameters

```
fn square(x: i16) -> i16 {  
    return x * x;  
}
```

2. Functions

Parameters

Return Type (optional)

```
fn square(x: i16) -> i16 {  
    return x * x;  
}
```

2. Functions

Parameters

Return Type (optional)

```
fn square(x: i16) -> i16 {  
    return x * x;  
}
```

Function body

2. Functions

Parameters

Return Type (optional)

```
fn square(x: i16) -> i16 {  
    return x * x;  
}
```

return keyword

→ we return the provided value (here `x * x`) from the function

2. Functions

```
fn square(x: i16) -> i16 {  
    return x * x;  
}
```

Function name

Parameters

Return Type (optional)

return keyword

→ we return the provided value (here `x * x`) from the function



2. Functions

```
fn many(mut a: i16, bla: u32, x: f32) {  
    a = 5;  
    println!("{}", bla);  
    println!("{}", x);  
}
```

2. Functions

Parameters are comma-separated name-type pairs

```
fn many(mut a: i16, bla: u32, x: f32) {  
    a = 5;  
    println!("{}", bla);  
    println!("{}", x);  
}
```

2. Functions

```
fn many(mut a: i16, bla: u32, x: f32) {  
    a = 5;  
    println!("{}", bla);  
    println!("{}", x);  
}
```

Parameters are **treated like normal variables**:

- Mutability rules apply
- Ownership rules apply
- Borrow Checker rules apply



2. Functions

Functions can also take zero parameters

```
fn none() {  
}
```



2. Functions

- Not all functions return values, some just do stuff with the inputs
 - It might just write to a file, or print something in the console



2. Functions

- Not all functions return values, some just do stuff with the inputs
 - It might just write to a file, or print something in the console
- The **return type is optional**, and declared using the **Arrow syntax**



2. Functions

- The **return type is optional**, and declared using the **Arrow syntax**

```
fn returns() -> i32 {  
    let a: i32 = 18;  
    return a;  
}
```

2. Functions

- The **return type is optional**, and declared using the **Arrow syntax**

Arrow Syntax

```
fn returns() -> i32 {  
    let a: i32 = 18;  
    return a;  
}
```



2. Functions

- The **return type is optional**, and declared using the **Arrow syntax**

This function **returns a value of type i32**

```
fn returns() -> i32 {  
    let a: i32 = 18;  
    return a;  
}
```



2. Functions

- The **return type is optional**, and declared using the **Arrow syntax**

```
fn returns() -> i32 {  
    let a: i32 = 18;  
    return a;  
}
```

All return statements **must return a value of the specified type**



2. Functions

```
fn invalid() -> i32 {  
    let a: i8 = 12;  
    return a;  
}
```

2. Functions

```
fn invalid() -> i32 {  
    let a: i8 = 12;  
    return a;  
}
```

returns a value of type i8
→ i8 is not i32
→ Compiler error



2. Functions

```
fn branch(a: i32) -> i32 {  
    if a < 0 {  
        println!("No return!");  
    } else {  
        return a;  
    }  
}
```

2. Functions

```
fn branch(a: i32) -> i32 {  
    if a < 0 {  
        println!("No return!");  
    } else {  
        return a;  
    }  
}
```

If a function is declared to return something, **every possible control flow path** must end with a return statement

2. Functions

```
fn branch(a: i32) -> i32 {  
    if a < 0 {  
        println!("No return!");  
    } else {  
        return a;  
    }  
}
```

This branch does not return a value,
not even after the if-block

If a function is declared to return
something, **every possible control flow
path** must end with a return statement

2. Functions

1/3

```
fn returns(a: i32) -> i32 {  
    if a < 0 { return 0; }  
    while a >= 0 {  
        if a == 5 { return 1; }  
        a -= 1;  
    }  
}
```

2. Functions

1/3

```
fn returns(a: i32) -> i32 {  
    if a < 0 { return 0; }  
    while a >= 0 {  
        if a == 5 { return 1; }  
        a -= 1;  
    }  
}
```

Do all possible paths lead to a return?

2. Functions

1/3

```
fn returns(a: i32) -> i32 {  
    if a < 0 { return 0; }  
    while a >= 0 {  
        if a == 5 { return 1; }  
        a -= 1;  
    }  
}
```

Do all possible paths lead to a return?

Nope. Imagine `a` was 4 in the beginning :^)

We get to the end of the function, but there is no return statement there!

2. Functions

1/3

```
fn returns(a: i32) -> i32 {  
    if a < 0 { return 0; }  
    while a >= 0 {  
        if a == 5 { return 1; }  
        a -= 1;  
    }  
}
```

Code wouldn't compile anyway, `a` is `not mutable` :^)



2. Functions

```
fn ret(mut a: i32) {  
    if a <= 0 { return; }  
    while a >= 0 {  
        if a == 5 { return; }  
        a -= 1;  
    }  
    println!("Hello!");  
}
```

2. Functions

We don't return anything

```
fn ret(mut a: i32)[] {  
    if a <= 0 { return; }  
    while a >= 0 {  
        if a == 5 { return; }  
        a -= 1;  
    }  
    println!("Hello!");  
}
```

2. Functions

```
fn ret(mut a: i32) {  
    if a <= 0 { return; }  
    while a >= 0 {  
        if a == 5 { return; }  
        a -= 1;  
    }  
    println!("Hello!");  
}
```

Now we simply return – without a value

2. Functions

```
fn ret(mut a: i32) {  
    if a <= 0 { return; }  
    while a >= 0 {  
        if a == 5 { return; }  
        a -= 1;  
    }  
    println!("Hello!");  
}
```

There's an **implicit return** at the end of every function,
not every control flow path needs to end in a return



2. Functions

```
fn call_this() {  
    println!("called call_this()!");  
}  
fn example1() {  
    call_this();  
}
```

2. Functions

```
fn call_this() {  
    println!("called call_this()!");  
}
```

Function declaration

```
fn example1() {  
    call_this();  
}
```

2. Functions

```
fn call_this() {  
    println!("called call_this()!");  
}
```

Function declaration

```
fn example1() {  
    call_this();  
}
```

Function call

2. Functions

```
fn call_this() {  
    println!("called call_this()!");  
}  
  
fn example1() {  
    call_this();  
}
```

For every **parameter**, we have to provide an **argument**
→ **call_this** has no parameters, so we can call it as is

2. Functions

```
fn print_arg(vec: Vec<i32>) {  
    println!("{:?}", vec);  
}  
  
fn example2() {  
    let v: Vec<u8> = vec![1, 2, 3];  
    print_arg(vec: v);  
}
```

2. Functions

```
fn print_arg(vec: Vec<i32>) {  
    println!("{:?}", vec);  
}  
  
fn example2() {  
    let v: Vec<u8> = vec![1, 2, 3];  
    print_arg(vec: v);  
}
```

Every argument has to have the same type as the matching parameter

2. Functions

```
fn print_arg(vec: Vec<i32>) {  
    println!("{:?}", vec);  
}
```

```
fn example2() {  
    let v: Vec<u8> = vec![1, 2, 3];  
    print_arg(vec: v);  
}
```

Type doesn't match → Compiler error

Every argument has to have the same type as the matching parameter

2. Functions

```
fn print_arg(vec: Vec<i32>) {  
    println!("{:?}", vec);  
}  
fn example2() {  
    let v: Vec<u8> = vec![1, 2, 3];  
    print_arg(vec: v);  
}
```

```
error[E0308]: mismatched types  
--> src\main.rs:14:15  
14 |         print_arg(v);  
    |         ^ expected `Vec<i32>`, found `Vec<u8>`  
    |  
    | arguments to this function are incorrect  
= note: expected struct `Vec<i32>`  
        found struct `Vec<u8>`  
note: function defined here  
--> src\main.rs:9:4  
9 | fn print_arg(vec: Vec<i32>) {  
  | ^^^^^^^^^^^ ^^^^^^^^^^^
```



2. Functions

```
fn get_vec() -> Vec<i32> {  
    return vec![1, 2, 3];  
}  
fn example3() {  
    let result: Vec<i32> = get_vec();  
    println!("{:?}", result);  
}
```

2. Functions

```
fn get_vec() -> Vec<i32> {  
    return vec![1, 2, 3];  
}
```

```
fn example3() {  
    let result: Vec<i32> = get_vec();  
    println!("{:?}", result);  
}
```

Return value can be used here

2. Functions

```
fn get_vec() -> Vec<i32> {  
    return vec![1, 2, 3];  
}
```

result in `example3` has this value after the function call

```
fn example3() {  
    let result: Vec<i32> = get_vec();  
    println!("{:?}", result);  
}
```

Running

[1, 2, 3]



2. Functions

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

2. Functions

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

Function declared to take 2 parameters

2. Functions

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}
```

Function declared to take 2 parameters
→ Function takes 2 arguments

```
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

2. Functions

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

Function declared to take 2 parameters
→ Function takes 2 arguments

Function declared to return `i32`

2. Functions

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

Function declared to take 2 parameters
→ Function takes 2 arguments

Function declared to return `i32`
→ `add(x, y)` evaluates to an `i32`

2. Functions

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}
```

Function declared to take 2 parameters
→ Function takes 2 arguments

```
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

Function declared to return `i32`
→ `add(x, y)` evaluates to an `i32`
→ `z` is of type `i32`

2. Functions

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}
```

Function declared to take 2 parameters
→ Function takes 2 arguments

```
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

Function declared to return `i32`
→ `add(x, y)` evaluates to an `i32`
→ `z` is of type `i32`
Code compiles, and prints 31

z=31

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}
```

But how does this work?

```
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

How does **a** know the value of **x**?
How does **b** know the value of **y**?
How does **z** know the **return value**?

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

To understand this, we need to revisit **memory and control flow**

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

To understand this, we need to revisit **memory and control flow**
→ Instructions are stored in memory

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

To understand this, we need to revisit **memory and control flow**
→ Instructions are stored in memory
→ Whenever we call a function, the computer **jumps there and executes that code**

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

To understand this, we need to revisit **memory and control flow**

- Instructions are stored in memory
- Whenever we call a function, the computer **jumps there and executes that code**
- **Variables and parameters are stored on the stack** (usually)

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

To understand this, we need to revisit **memory and control flow**

→ Instructions are stored in memory

→ Whenever we call a function, the computer **jumps there and executes that code**

→ **Variables and parameters are stored on the stack** (usually)

→ Every function **allocates its own stack region**, and **frees it when you leave**

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

Stack	

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

Stack	
x	???
y	???
z	???

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

Stack	
x	14
y	???
z	???

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

Stack	
x	14
y	17
z	???

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

Stack	
x	14
y	17
z	???

We want to jump to **add**, but we **need to pass the arguments**.
But we **can't assign** to **a** and **b** yet, they don't exist in memory!

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

We want to jump to **add**, but we **need to pass the arguments**.
But we **can't assign** to **a** and **b** yet, they don't exist in memory!

Solution: Our computer has **registers**

Stack	
x	14
y	17
z	???

Register 1	???
Register 2	???

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

We want to jump to **add**, but we **need to pass the arguments**.
But we **can't assign** to **a** and **b** yet, they don't exist in memory!

Solution: Our computer has **registers**

Stack	
x	14
y	17
z	???

Register 1	14
Register 2	???

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

We want to jump to **add**, but we **need to pass the arguments**.
But we **can't assign** to **a** and **b** yet, they don't exist in memory!

Solution: Our computer has **registers**

Stack	
x	14
y	17
z	???

Register 1	14
Register 2	17

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

We can now call the function

Stack	
x	14
y	17
z	???

Register 1	14
Register 2	17

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

Stack	
x	14
y	17
z	???
a	???
b	???

Register 1	14
Register 2	17

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

Stack	
x	14
y	17
z	???
a	???
b	???

We can now move the arguments into the variables

Register 1	14
Register 2	17

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

Stack	
x	14
y	17
z	???
a	14
b	???

We can now move the arguments into the variables

Register 1	14
Register 2	17

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

Stack	
x	14
y	17
z	???
a	14
b	17

We can now move the arguments into the variables

Register 1	14
Register 2	17

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

Stack	
x	14
y	17
z	???
a	14
b	17

We can now move the arguments into the variables
And execute the code in our function!

Register 1	14
Register 2	17

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

However, we don't know where we should store this result.
It may be **z**, it may be **in a vector**, we don't know!

Stack	
x	14
y	17
z	???
a	14
b	17

Register 1	14
Register 2	17

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

However, we don't know where we should store this result.
It may be **z**, it may be **in a vector**, we don't know!

Solution: **Store it in a register**, and **leave it to the caller**

Stack	
x	14
y	17
z	???
a	14
b	17

Register 1	31
Register 2	17

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

We return now, which means cleaning the stack

Stack	
x	14
y	17
z	???

Register 1	31
Register 2	17

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

Stack	
x	14
y	17
z	???

We now have the **result in Register 1**, which we can use

Register 1	31
Register 2	17

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

Stack	
x	14
y	17
z	31

We now have the **result in Register 1**, which we can use
→ We **move the value into z**

Register 1	31
Register 2	17

Intermission – Control Flow, again

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
fn example4() {  
    let x: i32 = 14;  
    let y: i32 = 17;  
    let z: i32 = add(a: x, b: y);  
    println!("z={}", z);  
}
```

Stack	
x	14
y	17
z	31

Our program prints 31

z=31

2. Functions

```
fn factorial(n: i32) -> i32 {  
    if n <= 1 {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}  
  
fn example5() {  
    let n: i32 = 10;  
    let result: i32 = factorial(n);  
    println!("{}", n, result);  
}
```

2. Functions

```
fn factorial(n: i32) -> i32 {  
    if n <= 1 {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}  
  
fn example5() {  
    let n: i32 = 10;  
    let result: i32 = factorial(n);  
    println!("{}", n, result);  
}
```

Function calls are expressions

2. Functions

```
fn factorial(n: i32) -> i32 {  
    if n <= 1 {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}  
  
fn example5() {  
    let n: i32 = 10;  
    let result: i32 = factorial(n);  
    println!("{}", n, result);  
}
```

Recursion: Calling a function in itself

2. Functions

Stack

```
fn factorial(n: i32) -> i32 {
    if n <= 1 {
        return 1;
    } else {
        return n * factorial(n - 1);
    }
}

fn example5() {
    let n: i32 = 10;
    let result: i32 = factorial(n);
    println!("{}", n, result);
}
```

2. Functions

```
fn factorial(n: i32) -> i32 {  
    if n <= 1 {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}  
fn example5() {  
    let n: i32 = 10;  
    let result: i32 = factorial(n);  
    println!("{}", n, result);  
}
```

Stack	
n	???
result	???

Registers	
Ret	???
Arg	???

2. Functions

```
fn factorial(n: i32) -> i32 {
    if n <= 1 {
        return 1;
    } else {
        return n * factorial(n - 1);
    }
}

fn example5() {
    let n: i32 = 10;
    let result: i32 = factorial(n);
    println!("{}", n, result);
}
```

Stack	
n	10
result	???

Registers	
Ret	???
Arg	???

2. Functions

```
fn factorial(n: i32) -> i32 {
    if n <= 1 {
        return 1;
    } else {
        return n * factorial(n - 1);
    }
}

fn example5() {
    let n: i32 = 10;
    let result: i32 = factorial(n);
    println!("{}", n, result);
}
```

Stack	
n	10
result	???

Registers	
Ret	???
Arg	10

2. Functions

```
fn factorial(n: i32) -> i32 {  
    if n <= 1 {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}  
  
fn example5() {  
    let n: i32 = 10;  
    let result: i32 = factorial(n);  
    println!("{}", n, result);  
}
```

Stack	
n	10
result	???
n	10

Registers	
Ret	???
Arg	10

2. Functions

```
fn factorial(n: i32) -> i32 {  
    if false {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}  
  
fn example5() {  
    let n: i32 = 10;  
    let result: i32 = factorial(n);  
    println!("{}", n, result);  
}
```

Stack	
n	10
result	???
n	10

Registers	
Ret	???
Arg	10

2. Functions

```
fn factorial(n: i32) -> i32 {  
    if false {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}  
  
fn example5() {  
    let n: i32 = 10;  
    let result: i32 = factorial(n);  
    println!("{}", n, result);  
}
```

Need to evaluate this expression before we can return a value

Stack	
n	10
result	???
n	10

Registers	
Ret	???
Arg	10

2. Functions

```
fn factorial(n: i32) -> i32 {  
    if false {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}  
  
fn example5() {  
    let n: i32 = 10;  
    let result: i32 = factorial(n);  
    println!("{}", n, result);  
}
```

Stack	
n	10
result	???
n	10

Registers	
Ret	???
Arg	9

2. Functions

```
fn factorial(n: i32) -> i32 {  
    if n <= 1 {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}  
  
fn example5() {  
    let n: i32 = 10;  
    let result: i32 = factorial(n);  
    println!("{}", n, result);  
}
```

Stack	
n	10
result	???
n	10
n	9

Registers	
Ret	???
Arg	9

2. Functions

```
fn factorial(n: i32) -> i32 {  
    if n <= 1 {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}  
  
fn example5() {  
    let n: i32 = 10;  
    let result: i32 = factorial(n);  
    println!("{}", n, result);  
}
```

Stack	
n	10
result	???
n	10
n	9

Registers	
Ret	???
Arg	9

Every time we call a function, we **allocate new stack space**
→ Every **n** we see here is **in a different memory location!**
→ Every function can only **directly access its own stack space**

2. Functions

```
fn factorial(n: i32) -> i32 {  
    if false {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}  
  
fn example5() {  
    let n: i32 = 10;  
    let result: i32 = factorial(n);  
    println!("{}", n, result);  
}
```

Stack	
n	10
result	???
n	10
n	9

Registers	
Ret	???
Arg	8

2. Functions

```
fn factorial(n: i32) -> i32 {  
    if n <= 1 {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}  
  
fn example5() {  
    let n: i32 = 10;  
    let result: i32 = factorial(n);  
    println!("{}", n, result);  
}
```

Stack	
n	10
result	???
n	10
n	9
n	8

Registers	
Ret	???
Arg	8

2. Functions

```
fn factorial(n: i32) -> i32 {  
    if false {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}  
  
fn example5() {  
    let n: i32 = 10;  
    let result: i32 = factorial(n);  
    println!("{}", n, result);  
}
```

Stack	
n	10
result	???
n	10
n	9
n	8

Registers	
Ret	???
Arg	7

2. Functions

```
fn factorial(n: i32) -> i32 {  
    if n <= 1 {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}  
  
fn example5() {  
    let n: i32 = 10;  
    let result: i32 = factorial(n);  
    println!("{}", n, result);  
}
```

Stack	
n	10
result	???
n	10
n	9
n	8
n	7

Registers	
Ret	???
Arg	7

2. Functions

```
fn factorial(n: i32) -> i32 {  
    if n <= 1 {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}  
  
fn example5() {  
    let n: i32 = 10;  
    let result: i32 = factorial(n);  
    println!("{}", n, result);  
}
```

Much later...

Stack	
n	10
result	???
n	10
n	9
n	8
n	7
n	6
n	5
n	4
n	3
n	2
n	1

Registers	
Ret	???
Arg	1

2. Functions

```
fn factorial(n: i32) -> i32 {  
    if n <= 1 {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}  
  
fn example5() {  
    let n: i32 = 10;  
    let result: i32 = factorial(n);  
    println!("{}", n, result);  
}
```

This is where the name **Stack Overflow** comes from:

- Every time we **call a recursive function, we allocate more memory**
- Memory is **limited, we can easily allocate too much**

Stack	
n	10
result	???
n	10
n	9
n	8
n	7
n	6
n	5
n	4
n	3
n	2
n	1

Registers	
Ret	???
Arg	1

2. Functions

```
fn factorial(n: i32) -> i32 {  
    if true {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}  
  
fn example5() {  
    let n: i32 = 10;  
    let result: i32 = factorial(n);  
    println!("{}", n, result);  
}
```

Stack	
n	10
result	???
n	10
n	9
n	8
n	7
n	6
n	5
n	4
n	3
n	2
n	1

Registers	
Ret	1
Arg	1

2. Functions

```
fn factorial(n: i32) -> i32 {
    if n <= 1 {
        return 1;
    } else {
        return n * factorial(n - 1);
    }
}
```

The computer keeps track of called functions

→ We can now go up the call chain and compute the result

```
fn example5() {
    let n: i32 = 10;
    let result: i32 = factorial(n);
    println!("{}", n, result);
}
```

Stack	
n	10
result	???
n	10
n	9
n	8
n	7
n	6
n	5
n	4
n	3
n	2

Registers	
Ret	1
Arg	1

2. Functions

```
fn factorial(n: i32) -> i32 {  
    if n <= 1 {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}  
fn example5() {  
    let n: i32 = 10;  
    let result: i32 = factorial(n);  
    println!("{}", n, result);  
}
```

The computer keeps track of called functions

→ We can now go up the call chain and compute the result

Stack	
n	10
result	???
n	10
n	9
n	8
n	7
n	6
n	5
n	4
n	3

Registers	
Ret	2
Arg	1

2. Functions

```
fn factorial(n: i32) -> i32 {  
    if n <= 1 {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}  
fn example5() {  
    let n: i32 = 10;  
    let result: i32 = factorial(n);  
    println!("{}", n, result);  
}
```

The computer keeps track of called functions

→ We can now go up the call chain and compute the result

Stack	
n	10
result	???
n	10
n	9
n	8
n	7
n	6
n	5
n	4

Registers	
Ret	6
Arg	1

2. Functions

```
fn factorial(n: i32) -> i32 {  
    if n <= 1 {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}  
fn example5() {  
    let n: i32 = 10;  
    let result: i32 = factorial(n);  
    println!("{}", n, result);  
}
```

The computer keeps track of called functions

→ We can now go up the call chain and compute the result

Stack	
n	10
result	???
n	10
n	9
n	8
n	7
n	6
n	5

Registers	
Ret	24
Arg	1

2. Functions

```
fn factorial(n: i32) -> i32 {  
    if n <= 1 {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}  
→ We can now go up the call chain and compute the result  
fn example5() {  
    let n: i32 = 10;  
    let result: i32 = factorial(n);  
    println!("{}", n, result);  
}
```

Much later...

Stack	
n	10
result	???
n	10

Registers	
Ret	362880
Arg	1

2. Functions

```
fn factorial(n: i32) -> i32 {
    if n <= 1 {
        return 1;
    } else {
        return n * factorial(n - 1);
    }
}

fn example5() {
    let n: i32 = 10;
    let result: i32 = factorial(n);
    println!("{}", n, result);
}
```

Stack	
n	10
result	???

Registers	
Ret	3628800
Arg	1

We have now computed the result, which is **located in the return register**

2. Functions

```
fn factorial(n: i32) -> i32 {
    if n <= 1 {
        return 1;
    } else {
        return n * factorial(n - 1);
    }
}

fn example5() {
    let n: i32 = 10;
    let result: i32 = factorial(n);
    println!("{}", n, result);
}
```

Stack	
n	10
result	3628800

Registers	
Ret	3628800
Arg	1

2. Functions

```
fn factorial(n: i32) -> i32 {
    if n <= 1 {
        return 1;
    } else {
        return n * factorial(n - 1);
    }
}

fn example5() {
    let n: i32 = 10;
    let result: i32 = factorial(n);
    println!("{}", n, result);
}
```

Stack	
n	10
result	3628800

Registers	
Ret	3628800
Arg	1

10! = 3628800



3. Floating Point Numbers



3. Floating Point Numbers

- Integers are good, your computer loves them, but they have their limits



3. Floating Point Numbers

- Integers are good, your computer loves them, but they have their limits
 - We can **only represent whole numbers**
 - We **can't represent big numbers**



3. Floating Point Numbers

- Integers are good, your computer loves them, but they have their limits
- For those situations, we have to use **floating point numbers**



3. Floating Point Numbers

- Integers are good, your computer loves them, but they have their limits
- For those situations, we have to use **floating point numbers**
- Rust has **f32** and **f64**



3. Floating Point Numbers

- Integers are good, your computer loves them, but they have their limits
- For those situations, we have to use **floating point numbers**
- Rust has **f32** and **f64**
- As with integers, the number **specifies the size in bits**
 - f32 → **32bit float**
 - f64 → **64bit float**



3. Floating Point Numbers

- Integers are good, your computer loves them, but they have their limits
- For those situations, we have to use **floating point numbers**
- Rust has **f32** and **f64**
- As with integers, the number **specifies the size in bits**
- Floating Point numbers are numbers in **Scientific Notation (in base 2)**
 - Example in base 10: $5e7 = 5 \cdot 10^7 = 50.000.000$



3. Floating Point Numbers

- Integers are good, your computer loves them, but they have their limits
- For those situations, we have to use **floating point numbers**
- Rust has **f32** and **f64**
- As with integers, the number **specifies the size in bits**
- Floating Point numbers are numbers in **Scientific Notation (in base 2)**
 - Example in base 10: $5e7 = 5 \cdot 10^7 = 50.000.000$
 - Numbers are stored in two parts:
 - **Mantissa** → Number before the **e**, here **5**
 - **Exponent** → Number after the **e**, here **7**



3. Floating Point Numbers

- Floating Point numbers are numbers in **Scientific Notation (in base 2)**
 - Example in base 10: $5e7 = 5 \cdot 10^7 = 50.000.000$
 - Numbers are stored in two parts:
 - **Mantissa** → Number before the **e**, here **5**
 - **Exponent** → Number after the **e**, here **7**
- Floats get a **few bits for the mantissa**, and a **few for the exponent**
 - f32 → **1 bit sign, 8 bits exponent , 23 bits mantissa**
 - f64 → **1 bit sign, 11 bits exponent , 52 bits mantissa**



3. Floating Point Numbers

- Floating Point numbers are numbers in **Scientific Notation (in base 2)**
- Floats get a **few bits for the mantissa**, and a **few for the exponent**
- This way, we can represent **fractions**
 - f32 → **~8 decimal digits**
 - f64 → **~16 decimal digits**



3. Floating Point Numbers

- Floating Point numbers are numbers in **Scientific Notation (in base 2)**
- Floats get a **few bits for the mantissa**, and a **few for the exponent**
- This way, we can represent **fractions**
 - f32 → **~8 decimal digits**
 - f64 → **~16 decimal digits**
- This way, we can represent **big numbers**
 - f32 → **$\sim 3.4 \cdot 10^{38} = 340\ldots000$** with 38 zeroes
 - With integers, we would need a **u128 to represent** that
 - f64 → **$\sim 1.8 \cdot 10^{308} = 180\ldots000$** with 308 zeroes
 - With integers, we would need a **u1024 to represent** that



3. Floating Point Numbers

- Floating Point numbers are numbers in **Scientific Notation (in base 2)**
- Floats get a **few bits for the mantissa**, and a **few for the exponent**
- This way, we can represent **fractions**
 - f32 $\rightarrow$ **~ 8 decimal digits**
 - f64 $\rightarrow$ **~ 16 decimal digits**
- This way, we can represent **big numbers**
 - f32 $\rightarrow$ **$\sim 3.4 \cdot 10^{38} = 340\ldots000$** with 38 zeroes
 - f64 $\rightarrow$ **$\sim 1.8 \cdot 10^{308} = 180\ldots000$** with 308 zeroes
- This way, we can represent **small numbers**
 - f32 $\rightarrow$ **$\sim 3.4 \cdot 10^{-38} = 0.000\ldots034$** with 38 zeroes
 - f64 $\rightarrow$ **$\sim 1.8 \cdot 10^{-308} = 0.000\ldots018$** with 308 zeroes



3. Floating Point Numbers

- There's a problem → Limited precision



3. Floating Point Numbers

- There's a problem → Limited precision
 - We can only store 8 (f64: 16) decimal digits at all times in the mantissa
 - At big exponents, we skip numbers
 - $5.400000000 \cdot 10^{30} + 1 == 5.400000000 \cdot 10^{30}$



3. Floating Point Numbers

- There's a problem → Limited precision
 - We can only store 8 (f64: 16) decimal digits at all times in the mantissa
 - At big exponents, we skip numbers
 - $5.400000000 \cdot 10^{30} + 1 == 5.400000000 \cdot 10^{30}$
 - We only have limited space for numbers
 - Just like we can't represent $1/3$ in base 10, we can't represent some numbers in base 2



3. Floating Point Numbers

- There's a problem → Limited precision
 - We can only store 8 (f64: 16) decimal digits at all times in the mantissa
 - At big exponents, we skip numbers
 - $5.400000000 \cdot 10^{30} + 1 == 5.400000000 \cdot 10^{30}$
 - We only have limited space for numbers
 - Just like we can't represent $1/3$ in base 10, we can't represent some numbers in base 2
- In practice, they are still extremely useful for applications where „good enough“ is fine
 - For example, Computer Graphics or Physics Simulations
 - Just don't manage your finances with them :^)

3. Floating Point Numbers

```
fn main() {  
    let a: f64 = 0.1;  
    let b: f64 = 0.2;  
    let c: f64 = a + b;  
    println!("a + b == {}", a + b);  
    println!("a + b == 0.3: {}", a + b == 0.3);  
    println!("      c == 0.3: {}", c == 0.3);  
    println!("c - b == a : {}", c - b == a);  
}
```

3. Floating Point Numbers

```
fn main() {  
    let a: f64 = 0.1;  
    let b: f64 = 0.2;  
    let c: f64 = a + b;  
    println!("a + b == {}", a + b);  
    println!("a + b == 0.3: {}", a + b == 0.3);  
    println!("      c == 0.3: {}", c == 0.3);  
    println!("c - b == a : {}", c - b == a);  
}
```

With Floating Point numbers, we can now use fractions

3. Floating Point Numbers

```
fn main() {  
    let a: f64 = 0.1;  
    let b: f64 = 0.2;  
    let c: f64 = a + b;  
    println!("a + b == {}", a + b);  
    println!("a + b == 0.3: {}", a + b == 0.3);  
    println!("c == 0.3: {}", c == 0.3);  
    println!("c - b == a : {}", c - b == a);  
}
```

However, even simple operations may not behave the way you would expect

3. Floating Point Numbers

```
fn main() {  
    let a: f64 = 0.1;  
    let b: f64 = 0.2;  
    let c: f64 = a + b;  
    println!("a + b == {}", a + b);  
    println!("a + b == 0.3: {}", a + b == 0.3);  
    println!("      c == 0.3: {}", c == 0.3);  
    println!("c - b == a : {}", c - b == a);  
}
```

```
a + b == 0.30000000000000000000000004  
a + b == 0.3: false  
      c == 0.3: false  
c - b == a : false
```

3. Floating Point Numbers

```
fn main() {  
    let a: f64 = 0.1;  
    let b: f64 = 0.2;  
    let c: f64 = a + b;  
    println!("a + b == {}", a + b);  
    println!("a + b == 0.3: {}", a + b == 0.3);  
    println!("      c == 0.3: {}", c == 0.3);  
    println!("c - b == a : {}", c - b == a);  
}
```

`a + b == 0.300000000000000000000004`

`a + b == 0.3: false`

`c == 0.3: false`

`c - b == a : false`

Floating Point **can't represent every number**,
we have to **deal with precision errors**

3. Floating Point Numbers

```
fn main() {  
    let a: f64 = 0.1;  
    let b: f64 = 0.2;  
    let c: f64 = a + b;  
    println!("a + b == {}", a + b);  
    println!("a + b == 0.3: {}", a + b == 0.3);  
    println!("      c == 0.3: {}", c == 0.3);  
    println!("c - b == a   : {}", c - b == a);  
    let approx: bool = f64::abs(self: (a + b) - c) < f64::EPSILON;  
    println!("close enough: {}", approx);  
}
```

3. Floating Point Numbers

```
fn main() {  
    let a: f64 = 0.1;  
    let b: f64 = 0.2;  
    let c: f64 = a + b;  
    println!("a + b == {}", a + b);  
    println!("a + b == 0.3: {}", a + b == 0.3);  
    println!("      c == 0.3: {}", c == 0.3);  
    println!("c - b == a   : {}", c - b == a);  
    let approx: bool = f64::abs(self: (a + b) - c) < f64::EPSILON;  
    println!("close enough: {}", approx);  
}
```

Always necessary to test for equality within a given tolerance
→ The numbers are close enough and effectively the same

3. Floating Point Numbers

```
fn main() {  
    let a: f64 = 0.1;  
    let b: f64 = 0.2;  
    let c: f64 = a + b;  
    println!("a + b == {}", a + b);  
    println!("a + b == 0.3: {}", a + b == 0.3);  
    println!("      c == 0.3: {}", c == 0.3);  
    println!("c - b == a   : {}", c - b == a);  
    let approx: bool = f64::abs(self: (a + b) - c) < f64::EPSILON;  
    println!("close enough: {}", approx);  
}
```

```
a + b == 0.30000000000000000000000000000004  
a + b == 0.3: false  
      c == 0.3: false  
c - b == a   : false  
close enough: true
```

3. Floating Point Numbers

```
fn main() {  
    let mass: f64 = 5.972168e27; // g  
    let radius: f64 = 637_100_000.0; // cm  
    let volume: f64 = 4.0 / 3.0 * PI * radius.powi(3);  
    let density: f64 = mass / volume; // g/cm^3  
    println!("Earth's density: {:.?}", density);  
}
```

3. Floating Point Numbers

```
fn main() {  
    let mass: f64 = 5.972168e27; // g  
    let radius: f64 = 637_100_000.0; // cm  
    let volume: f64 = 4.0 / 3.0 * PI * radius.powi(3);  
    let density: f64 = mass / volume; // g/cm^3  
    println!("Earth's density: {:?}", density);  
}
```

Float Literals can also be specified in scientific notation

3. Floating Point Numbers

```
fn main() {  
    let mass: f64 = 5.972168e27; // g  
    let radius: f64 = 637_100_000.0; // cm  
    let volume: f64 = 4.0 / 3.0 * PI * radius.powi(3);  
    let density: f64 = mass / volume; // g/cm^3  
    println!("Earth's density: {:?}", density);  
}
```

The **.0** is mandatory. Otherwise, Rust will treat this as an **integer literal** and **print an error**.

3. Floating Point Numbers

```
fn main() {  
    let mass: f64 = 5.972168e27; // g  
    let radius: f64 = 637_100_000.0; // cm  
    let volume: f64 = 4.0 / 3.0 * PI * radius.powi(3);  
    let density: f64 = mass / volume; // g/cm^3  
    println!("Earth's density: {:?}", density);  
} Earth's density: 5.51341383361054
```

3. Floating Point Numbers

```
fn main() {  
    let mass: f64 = 5.972168e27; // g  
    let radius: f64 = 637_100_000.0; // cm  
    let volume: f64 = 4.0 / 3.0 * PI * radius.powi(3);  
    let density: f64 = mass / volume; // g/cm^3  
    println!("Earth's density: {:.?}", density);  
} Earth's density: 5.51341383361054
```

Even with precision errors, floats are still **good enough for many applications**
Here: A **rounding error of 0.000000000001** doesn't influence the final result

But still, **errors propagate!** Calculation errors in a loop **may add up over time.**



Intermission - Exercises

- Time for exercises!

Intermission - Exercises

2/3

```
fn loop(start: i32, end: i32, mut counter: i32) {  
    if counter <= start {  
        counter = start;  
        return;  
    }  
    while counter < end {  
        println!("{}", counter);  
        counter += 1;  
    }  
    return counter;  
}
```

Intermission - Exercises

Is this a valid function definition?

2/3

```
fn loop(start: i32, end: i32, mut counter: i32) {  
    if counter <= start {  
        counter = start;  
        return;  
    }  
    while counter < end {  
        println!("{}", counter);  
        counter += 1;  
    }  
    return counter;  
}
```

Intermission - Exercises

Is this a valid function definition?

2/3

```
fn loop(start: i32, end: i32, mut counter: i32) {}  
    if counter <= start {  
        counter = start;  
        return;  
    }  
    while counter < end {  
        println!("{}", counter);  
        counter += 1;  
    }  
    return counter;  
}
```

Function not declared to return anything

Intermission - Exercises

Is this a valid function definition?

2/3

```
fn loop(start: i32, end: i32, mut counter: i32) {}  
    if counter <= start {  
        counter = start;  
        return;  
    }  
    while counter < end {  
        println!("{}", counter);  
        counter += 1;  
    }  
    return counter;  
}
```

Function not declared to return anything

Yet, we're attempting to return i32.
→ Not a valid function

Intermission - Exercises

2/3

```
fn contains(vec: &Vec<i32>, value: i32) -> bool {  
    for i: &i32 in vec {  
        if i == value {  
            return true;  
        }  
    }  
    return false;  
}  
  
▶ Run | Debug  
fn main() {  
    let v: Vec<i32> = vec![1, 2, 3];  
    let res: bool = contains(vec: &v, value: 2);  
    println!("res={}", res);  
}
```

Intermission - Exercises

2/3

```
fn contains(vec: &Vec<i32>, value: i32) -> bool {  
    for i: &i32 in vec {  
        if i == value {  
            return true;  
        }  
    }  
    return false;  
}  
  
▶ Run | Debug  
fn main() {  
    let v: Vec<i32> = vec![1, 2, 3];  
    let res: bool = contains(vec: &v, value: 2);  
    println!("res={}", res);  
}
```

Does the code compile?
If yes, what does it print?

Intermission - Exercises

2/3

```
fn contains(vec: &Vec<i32>, value: i32) -> bool {  
    for i: &i32 in vec {  
        if i == value {  
            return true;  
        }  
    }  
    return false;  
}
```

Does the code compile?
If yes, what does it print?

This code is almost perfect, there are no errors in the arguments or return values

► Run | Debug

```
fn main() {  
    let v: Vec<i32> = vec![1, 2, 3];  
    let res: bool = contains(vec: &v, value: 2);  
    println!("res={}", res);  
}
```

Intermission - Exercises

2/3

```
fn contains(vec: &Vec<i32>, value: i32) -> bool {  
    for i: &i32 in vec {  
        if i == value {  
            return true;  
        }  
    }  
    return false;  
}
```

▶ Run | Debug

```
fn main() {  
    let v: Vec<i32> = vec![1, 2, 3];  
    let res: bool = contains(vec: &v, value: 2);  
    println!("res={}", res);  
}
```

Does the code compile?
If yes, what does it print?

BUT: `&i32 == i32` is not defined :^)

The correct implementation would've been `*i == value`

This code is almost perfect, there are no errors in the arguments or return values

Intermission - Exercises

2/3

```
fn contains(vec: &Vec<i32>, value: i32) -> bool {  
    for i: &i32 in vec {  
        if i == value {  
            return true;  
        }  
    }  
    return false;  
}  
  
fn main() {  
    let v: Vec<i32> = vec![1, 2, 3];  
    let res: bool = contains(&v, 2);  
    println!("res={}", res);  
}
```

Does the code compile?
If yes, what does it print?

Intermission - Exercises

3/3

```
fn is_prime(candidate: i32) -> bool {  
    for number: i32 in 2..candidate {  
        if candidate % number == 0 {  
            return false;  
        }  
    }  
    return true;  
}
```

Intermission - Exercises

Is this function *correct*?

3/3

```
fn is_prime(candidate: i32) -> bool {  
    for number: i32 in 2..candidate {  
        if candidate % number == 0 {  
            return false;  
        }  
    }  
    return true;  
}
```

Intermission - Exercises

Is this function *correct*?

3/3

```
fn is_prime(candidate: i32) -> bool {  
    for number: i32 in 2..candidate {  
        if candidate % number == 0 {  
            return false;  
        }  
    }  
    return true;  
}
```

The algorithm itself is correct, it correctly tells us if a given number is prime or not.

Intermission - Exercises

Is this function *correct*?

3/3

```
fn is_prime(candidate: i32) -> bool {  
    for number: i32 in 2..candidate {  
        if candidate % number == 0 {  
            return false;  
        }  
    }  
    return true;  
}
```

The algorithm itself is correct, it correctly tells us if a given number is prime or not.

BUT: It returns true for negative numbers, 0 and 1! :^)

Those are not prime numbers :(

Intermission - Exercises

3/3

```
fn is_prime(candidate: i32) -> bool {  
    for number: i32 in 2..candidate {  
        if candidate % number == 0 {  
            return false;  
        }  
    }  
    return true;  
}
```

The algorithm itself is correct, it correctly tells us if a given number is prime or not.

BUT: It returns true for negative numbers, 0 and 1! :^)

Those are not prime numbers :(

How do we fix that?

Intermission - Exercises

Step 1: Prevent negative numbers!

3/3

```
fn is_prime(candidate: u32) -> bool {  
    for number: u32 in 2..candidate {  
        if candidate % number == 0 {  
            return false;  
        }  
    }  
    return true;  
}
```

Intermission - Exercises

3/3

```
fn is_prime(candidate: u32) -> bool {  
    if candidate < 2 { return false; }  
    for number: u32 in 2..candidate {  
        if candidate % number == 0 {  
            return false;  
        }  
    }  
    return true;  
}
```

Step 2: 0 and 1 are not prime

Intermission - Exercises

3/3

```
fn is_prime(candidate: u32) -> bool {  
    if candidate < 2 { return false; }  
    for number: u32 in 2..candidate {  
        if candidate % number == 0 {  
            return false;  
        }  
    }  
    return true;  
}
```

Is this a correct function now?

Intermission - Exercises

3/3

```
fn is_prime(candidate: u32) -> bool {  
    if candidate < 2 { return false; }  
    for number: u32 in 2..candidate {  
        if candidate % number == 0 {  
            return false;  
        }  
    }  
    return true;  
}
```

Is this a correct function now?

Yes :)



4. Next time

- Structs
- Methods